

CSAI415 Project — Deliverable 2 (D2)

Team

- Pair A: Ahmad Fraij, Yousef Alsakkaf
- Pair B: WAFIQ Akram ABO DAKEN, Ahmed Soliman
- Pair C: Abdurlahman Alali, Yehia Nouredin
- Solo: Musab

1) Objective and Scope

Deliverable 2 productionizes the D1 retriever into a runnable data stack with persistent storage, a retrieval API, and a graph backend. The work covers:

- Ingest and storage in MongoDB + Qdrant
- Hybrid retrieval service exposed through FastAPI
- Graph build in Neo4j with reusable Cypher queries
- Engineering glue (docker-compose, seed scripts, health checks, smoke tests)

D2 emphasizes integration and operational readiness, while retrieval quality tuning remains anchored to D1 and validated through dedicated metric tasks.

2) Architecture Summary

High-level dataflow:

``SciFact + arXiv PDFs -> ingest/chunk/embed -> Mongo + Qdrant + Neo4j -> FastAPI /search -> user``

Artifacts:

- Diagram source: ``reports/d2_dataflow.mmd``
- Rendered diagram: ``reports/d2_dataflow.png``

Storage contracts:

- Mongo (``csai415``):
 - ``papers`` collection for paper-level metadata (``title``, ``authors``, ``year``, ``topics``, ``source``)
 - ``chunks`` collection for retrieval units with provenance (``paper_id``, ``position``, page bounds, ``source``)
- Qdrant:
 - ``chunks_bge384`` (full)
 - ``chunks_bge384_scifact`` (source-specific)
 - ``chunks_bge384_arxiv`` (source-specific)
 - 384-dimensional vectors, cosine distance for ANN stage
- Neo4j:
 - Nodes: ``Paper``, ``Author``, ``Topic`` (+ ``Venue`` when available)
 - Edges: ``WROTE``, ``ABOUT``, ``PUBLISHED_IN``

Schema references:

- ``docs/d2_schemas.md``
- ``docs/d2_graph_schema.md``

3) Ingest and Storage (Rubric: 3%)

Implemented components:

- `scripts/seed\_mongo.py`
- `scripts/seed\_qdrant.py`
- `scripts/seed\_neo4j.py`
- Supporting ingestion utilities in `src/csai415/ingest.py`

Key outcomes:

- Idempotent seeding flows for repeatable local integration
- Provenance-preserving schema across papers/chunks/vector payloads
- Source-aware data organization for SciFact evaluation and arXiv graph/search demos

Verification evidence is documented in:

- `reports/d2\_int1\_verification.md`
- `reports/d2\_int2\_verification.md`

4) Hybrid Retrieval API (Rubric: 5%)

Service implementation:

- `src/csai415/api.py`
- Endpoints:
 - `POST /search`
 - `GET /healthz`

API contract:

- Request: `{ query: str, k: int = 5, source: str | null = null }`
- Response hit fields: `{ chunk\_id, paper\_id, title, text, page\_range, score }`

Design notes:

- Service loads blessed retrieval configuration from `configs/winning\_runcard.yaml`
- Source filtering is supported for `scifact`/`arxiv` use-cases
- Qdrant-backed dense retrieval wiring was integrated with startup-time sanity checks

Evaluation artifacts:

- `reports/d2\_search\_metrics.csv`
- `reports/d2\_topk\_examples.md`

5) Graph Build and Querying (Rubric: 5%)

Neo4j graph build:

- Loader script: `scripts/seed\_neo4j.py`
- Merge-based write semantics and constraints for safe reseeding
- Author-paper-topic topology aligned with D3 GraphRAG needs

Cypher deliverables:

- Query library in `cypher/`
- Captured examples in `reports/d2\_cypher\_examples.md`

Covered query patterns:

1. Papers by a specific author
2. Co-author ranking
3. Topic volume by year range
4. Authors and papers for a topic
5. Authors active in both of two topics

6) Engineering and Reproducibility (Rubric: 2%)

Operational deliverables:

- `docker-compose.yml` for Mongo, Qdrant, Neo4j, and API
- `.env.example` for standardized local configuration
- `Makefile` targets for stack lifecycle and seeding

Smoke testing:

- Existing D1 smoke flow remains green by default
- Added gated live-stack smoke in `tests/test\_smoke.py`:
 - `test\_d2\_stack\_smoke` (guarded by `D2\_STACK\_UP=1`)
 - Validates FastAPI, Mongo, Qdrant, and Neo4j reachability and seeded state

DoD wording in `D2\_TASKS.md` was updated to make both expected modes explicit:

- default: `11 passed, 1 skipped`
- with `D2\_STACK\_UP=1` and running stack: `12 passed`

7) Key Technical Decisions and Tradeoffs

1. Smoke tests check stack health, not ranking quality.
Quality metrics are tracked in dedicated evaluation artifacts to avoid fragile/flaky CI assertions.
2. Direct localhost integration checks were preferred over compose-managed fixtures.
This keeps runtime/setup overhead low and mirrors actual developer usage against `docker compose up`.
3. Dev-mode auth assumptions are explicit.
Neo4j dev setup uses `NEO4J\_AUTH=None`; production deployment should move credentials to environment configuration.
4. Thresholds are intentionally loose but meaningful.
Count checks detect unseeded or partially seeded states while avoiding unnecessary failures as collection sizes evolve.
5. Per-source Qdrant collection existence is enforced.
Missing source collections are surfaced immediately in smoke tests instead of failing later as opaque API runtime errors.

8) Risks and Mitigations

- Partial reseed risk: mitigated through explicit collection/node/count assertions.
- Source contamination risk in evaluation: mitigated via source-filter path and checks.
- Environment drift risk: mitigated by compose + env template + health and smoke tests.
- Metadata incompleteness risk in graph: documented and deferred for D3 expansion where applicable.

9) Deliverables Mapping

- Ingest and storage scripts + schema docs: complete
- Hybrid retrieval API endpoints + source filtering: complete
- Metrics table + top-k examples: complete
- Neo4j graph + Cypher query set + example outputs: complete
- Engineering bundle (compose, env, dataflow, smoke): complete

10) Conclusion

D2 successfully transitions the project from D1 experimental retrieval into an integrated, service-oriented system with persistent stores, vector search, graph support, and operational smoke coverage. This establishes a stable foundation for D3 GraphRAG work and further production hardening.